4. HotStuff 동기화 메커니즘

4.1 동기화의 필요성

4.1.1 뷰 동기화 문제

HotStuff에서 모든 복제본은 같은 "뷰(view)"에서 작업해야 합니다:

- **뷰**: 합의의 라운드를 나타내는 논리적 시간
- **리더**: 각 뷰마다 하나의 리더가 블록을 제안
- **진행**: 모든 복제본이 동일한 뷰에서 진행해야 활성 보장

4.1.2 동기화 실패 시나리오

```
복제본 1: 뷰 5에서 대기 중
복제본 2: 뷰 7에서 작업 중
복제본 3: 뷰 6에서 타임아웃
→ 합의 진행 불가능
```

4.2 Synchronizer 구조

4.2.1 핵심 구조체

```
type Synchronizer struct {
// 의존성 모듈들
```

```
blockChain modules.BlockChain

consensus modules.Consensus

crypto modules.Crypto

configuration modules.Configuration

eventLoop *eventloop.EventLoop

leaderRotation modules.LeaderRotation

logger logging.Logger

opts *modules.Options

// 동기화 상태 (뮤텍스로 보호)

mut sync.RWMutex

currentView hotstuff.View // 현재 뷰

highTC hotstuff.TimeoutCert // 최고 타임아웃 인증서

highQC hotstuff.QuorumCert // 최고 쿼럼 인증서

// 타임아웃 관리

lastTimeout *hotstuff.TimeoutMsg // 마지막 타임아웃 메시지

duration ViewDuration // 뷰 지속 시간 관리

timer oneShotTimer // 타임아웃 타이머

// 타임아웃 메시지 수집

timeouts  map[hotstuff.View]map[hotstuff.ID]hotstuff.TimeoutMsg

}
```

4.2.2 초기화 과정

```go
func (s *Synchronizer) InitModule(mods *modules.Core) {
// 의존성 주입
mods.Get(&s.blockChain, &s.consensus, &s.crypto, ...)
// 이벤트 핸들러 등록
s.eventLoop.RegisterHandler(TimeoutEvent{}, s.handleTimeoutEvent)
s.eventLoop.RegisterHandler(hotstuff.NewViewMsg{}, s.handleNewViewMsg)
s.eventLoop.RegisterHandler(hotstuff.TimeoutMsg{}, s.handleTimeoutMsg)
// 제네시스 QC/TC 생성
s.highQC = s.createGenesisQC()
s.highTC = s.createGenesisTC()
}
```

4.3 뷰 진행 메커니즘

4.3.1 정상적인 뷰 진행

```go
func (s *Synchronizer) AdvanceView(syncInfo hotstuff.SyncInfo) {
newView := s.calculateNewView(syncInfo)
if  newView <= s.View() {
return  // 이미 더 높은 뷰에 있음
}
// 타이머 중지
s.stopTimeoutTimer()
```

```
// 뷰 업데이트
s.currentView = newView
s.lastTimeout = nil
// 새 타이머 시작
s.duration.ViewStarted()
s.startTimeoutTimer()
s.logger.Debugf("advanced to view %d", newView)
// 뷰 변경 이벤트 발생
s.eventLoop.AddEvent(ViewChangeEvent{
View: newView,
Timeout: s.wasTimeout(syncInfo)
})
// 리더 결정 및 행동
s.handleLeaderAction(newView, syncInfo)
}
```

4.3.2 새 뷰 계산

```
func (s *Synchronizer) calculateNewView(syncInfo hotstuff.SyncInfo) hotstuff.Vie
v := hotstuff.View(0)
timeout := false
// 타임아웃 인증서 확인
if  tc, ok := syncInfo.TC();  ok {
```

```
if !s.crypto.VerifyTimeoutCert(tc) {
s.logger.Info("Timeout Certificate could not be verified!")
return  s.View()
}
s.updateHighTC(tc)
v = tc.View()
timeout = true
}
// 쿼럼 인증서 확인
if  qc, ok := syncInfo.QC();  ok {
if !s.crypto.VerifyQuorumCert(qc) {
s.logger.Info("Quorum Certificate could not be verified!")
return  s.View()
}
s.updateHighQC(qc)
// QC가 TC보다 높으면 QC 사용
if  qc.View() >= v {
v = qc.View()
timeout = false
}
}
// AggregateQC 처리 (Fast-HotStuff)
if  aggQC, ok := syncInfo.AggQC();  ok && s.opts.ShouldUseAggQC() {
```

```
highQC, valid := s.crypto.VerifyAggregateQC(aggQC)
if valid {
s.updateHighQC(highQC)
if aggQC.View() >= v {
v = aggQC.View()
timeout = true
}
}
}
return v + 1 // 다음 뷰로 진행
}
```

4.4 타임아웃 처리

4.4.1 로컬 타임아웃 발생

```
func (s *Synchronizer) OnLocalTimeout() {
view := s.View()
// 새 타이머 시작 (다음 타임아웃 대비)
s.startTimeoutTimer()
// 이미 같은 뷰에서 타임아웃했으면 재전송
if s.lastTimeout != nil && s.lastTimeout.View == view {
s.configuration.Timeout(*s.lastTimeout)
```

```
return
}
// 뷰 지속 시간 증가 (지수 백오프)
s.duration.ViewTimeout()
s.logger.Debugf("OnLocalTimeout: %v", view)
// 타임아웃 메시지 생성
timeoutMsg := s.createTimeoutMessage(view)
s.lastTimeout = &timeoutMsg
// 현재 뷰에서 투표 중지
s.consensus.StopVoting(view)
// 타임아웃 메시지 전파
s.configuration.Timeout(timeoutMsg)
s.OnRemoteTimeout(timeoutMsg) // 자신의 타임아웃도 처리
}
```

4.4.2 타임아웃 메시지 생성

```
func (s *Synchronizer) createTimeoutMessage(view hotstuff.View) hotstuff.Timeout
// 뷰에 대한 서명 생성
sig, err := s.crypto.Sign(view.ToBytes())
if  err != nil {
s.logger.Warnf("Failed to sign view: %v", err)
return hotstuff.TimeoutMsg{}
```

```
}
timeoutMsg := hotstuff.TimeoutMsg{
ID: s.opts.ID(),
View: view,
SyncInfo: s.SyncInfo(),
ViewSignature: sig,
}
// Fast-HotStuff용 추가 서명
if  s.opts.ShouldUseAggQC() {
msgSig, err := s.crypto.Sign(timeoutMsg.ToBytes())
if  err != nil {
s.logger.Warnf("Failed to sign timeout message: %v", err)
} else {
timeoutMsg.MsgSignature = msgSig
}
}
return  timeoutMsg
}
```

4.4.3 원격 타임아웃 처리

```
func (s *Synchronizer) OnRemoteTimeout(timeout hotstuff.TimeoutMsg) {
currView := s.View()
```

```
// 서명 검증
if !s.crypto.Verify(timeout.ViewSignature, timeout.View.ToBytes()) {
s.logger.Warn("Invalid timeout signature")
return
}
s.logger.Debug("OnRemoteTimeout: ", timeout)
// SyncInfo로 뷰 진행 시도
s.AdvanceView(timeout.SyncInfo)
// 타임아웃 메시지 수집
s.collectTimeoutMessage(timeout)
// 쿼럼 달성 시 타임아웃 인증서 생성
if  s.hasTimeoutQuorum(timeout.View) {
s.createTimeoutCertificate(timeout.View)
}
// 오래된 타임아웃 정리
s.cleanupOldTimeouts(currView)
}
```

4.4.4 타임아웃 인증서 생성

```
func (s *Synchronizer) createTimeoutCertificate(view hotstuff.View) {
timeouts := s.timeouts[view]
// 타임아웃 메시지 리스트 생성
```

```go
timeoutList := make([]hotstuff.TimeoutMsg, 0, len(timeouts))
for _, t := range timeouts {
timeoutList = append(timeoutList, t)
}
// 타임아웃 인증서 생성
tc, err := s.crypto.CreateTimeoutCert(view, timeoutList)
if err != nil {
s.logger.Debugf("Failed to create timeout certificate: %v", err)
return
}
si := s.SyncInfo().WithTC(tc)
// Fast-HotStuff용 AggregateQC 생성
if s.opts.ShouldUseAggQC() {
if aggQC, err := s.crypto.CreateAggregateQC(view, timeoutList); err == nil {
si = si.WithAggQC(aggQC)
}
}
// 타임아웃 메시지 정리
delete(s.timeouts, view)
// 새 뷰로 진행
s.AdvanceView(si)
}
```

4.5 타이머 관리

4.5.1 원샷 타이머 구조

```
type oneShotTimer struct {
timerDoNotUse *time.Timer
}

func (t oneShotTimer) Stop() bool {
return  t.timerDoNotUse.Stop()
}
```

설계 이유:

- 타이머 재사용 시 잘못된 뷰 번호 문제 방지
- 각 뷰마다 새로운 타이머 생성
- 메모리 누수 방지

4.5.2 타이머 시작

```
func (s *Synchronizer) startTimeoutTimer() {
// 현재 뷰를 로컬 변수에 저장 (데이터 레이스 방지)
view := s.View()
// 새 타이머 생성 (재사용 금지)
s.timer = oneShotTimer{
time.AfterFunc(s.duration.Duration(), func() {
```

```
// 이벤트 루프를 통해 타임아웃 처리
s.eventLoop.AddEvent(TimeoutEvent{view})
})
}
}
```

4.5.3 뷰 지속 시간 관리

```
type ViewDuration interface {
Duration() time.Duration // 현재 뷰 지속 시간
ViewStarted() // 새 뷰 시작 알림
ViewSucceeded() // 뷰 성공 알림
ViewTimeout() // 뷰 타임아웃 알림
}

// 고정 지속 시간 구현
type FixedViewDuration struct {
duration time.Duration
}

func (fvd FixedViewDuration) Duration() time.Duration {
return  fvd.duration
}
```

```go
// 적응적 지속 시간 구현 (지수 백오프)
type AdaptiveViewDuration struct {
baseDuration time.Duration
currentDuration time.Duration
multiplier  float64
maxDuration time.Duration
}

func (avd *AdaptiveViewDuration) ViewTimeout() {
// 지수적으로 증가
avd.currentDuration = time.Duration(
float64(avd.currentDuration) * avd.multiplier
)
if  avd.currentDuration > avd.maxDuration {
avd.currentDuration = avd.maxDuration
}
}

func (avd *AdaptiveViewDuration) ViewSucceeded() {
// 성공 시 기본값으로 리셋
avd.currentDuration = avd.baseDuration
```

```
}
```

4.6 리더 행동 결정

4.6.1 리더 결정 및 행동

```
func (s *Synchronizer) handleLeaderAction(newView hotstuff.View, syncInfo hotstu

leader := s.leaderRotation.GetLeader(newView)

if  leader == s.opts.ID() {

// 자신이 리더인 경우: 제안 생성

s.consensus.Propose(syncInfo)

} else {

// 다른 복제본이 리더인 경우: NewView 메시지 전송

if  replica, ok := s.configuration.Replica(leader);  ok {

replica.NewView(syncInfo)

} else {

s.logger.Warnf("Leader replica %d not found", leader)

}

}

}
```

4.6.2 NewView 메시지 처리

```go
func (s *Synchronizer) OnNewView(newView hotstuff.NewViewMsg) {
s.logger.Debugf("OnNewView from %d: %v", newView.ID, newView.SyncInfo)
// SyncInfo를 사용하여 뷰 진행 시도
s.AdvanceView(newView.SyncInfo)
}
```

4.7 동기화 정보 관리

4.7.1 SyncInfo 생성

```go
func (s *Synchronizer) SyncInfo() hotstuff.SyncInfo {
s.mut.RLock()
defer  s.mut.RUnlock()
return  hotstuff.NewSyncInfo().
WithQC(s.highQC).
WithTC(s.highTC)
}
```

4.7.2 최고 QC/TC 업데이트

```go
func (s *Synchronizer) updateHighQC(qc hotstuff.QuorumCert) {
// 블록 존재 확인
newBlock, ok := s.blockChain.Get(qc.BlockHash())
```

```
if !ok {
s.logger.Info("updateHighQC: Could not find block referenced by new QC!")
return
}
// 더 높은 뷰의 QC만 업데이트
if  newBlock.View() > s.highQC.View() {
s.highQC = qc
s.logger.Debug("HighQC updated")
}
}

func (s *Synchronizer) updateHighTC(tc hotstuff.TimeoutCert) {
if  tc.View() > s.highTC.View() {
s.highTC = tc
s.logger.Debug("HighTC updated")
}
}
```

4.8 시작 및 종료

4.8.1 동기화 시작

```
func (s *Synchronizer) Start(ctx context.Context) {
```

```
// 첫 번째 타이머 시작
s.startTimeoutTimer()
// 컨텍스트 취소 시 타이머 정리
go func() {
<-ctx.Done()
s.stopTimeoutTimer()
}()
// 초기 제안 (뷰 1의 리더인 경우)
if view := s.View(); view == 1 && s.leaderRotation.GetLeader(view) == s.opts.I
s.consensus.Propose(s.SyncInfo())
}
}
```

4.8.2 이벤트 처리

```
// 타임아웃 이벤트 처리
func (s *Synchronizer) handleTimeoutEvent(event any) {
timeoutView := event.(TimeoutEvent).View
if s.View() == timeoutView {
s.OnLocalTimeout()
}
}
```

```go
// NewView 메시지 처리
func (s *Synchronizer) handleNewViewMsg(event any) {
newViewMsg := event.(hotstuff.NewViewMsg)
s.OnNewView(newViewMsg)
}

// 타임아웃 메시지 처리
func (s *Synchronizer) handleTimeoutMsg(event any) {
timeoutMsg := event.(hotstuff.TimeoutMsg)
s.OnRemoteTimeout(timeoutMsg)
}
```

4.9 동기화 보장

4.9.1 안전성 보장

- **QC/TC 검증**: 모든 인증서의 서명과 쿼럼 크기 확인
- **뷰 단조성**: 뷰 번호는 항상 증가만 함
- **원자적 업데이트**: 뷰 변경은 원자적으로 수행

4.9.2 활성 보장

- **타임아웃 메커니즘**: 진행이 멈추면 자동으로 다음 뷰로 이동
- **지수 백오프**: 네트워크 상황에 따라 타임아웃 시간 조정
- **리더 교체**: 각 뷰마다 다른 리더로 교체

4.9.3 성능 최적화

- **타임아웃 메시지 캐싱**: 같은 뷰에서 재전송 시 캐시 사용
- **오래된 데이터 정리**: 메모리 누수 방지를 위한 주기적 정리
- **배치 처리**: 여러 타임아웃 메시지를 한 번에 처리

이러한 동기화 메커니즘을 통해 HotStuff는 네트워크 지연이나 일시적 장애가 있어도 모든 복제본이 동일한 뷰에서 작업하며 합의를 진행할 수 있습니다.